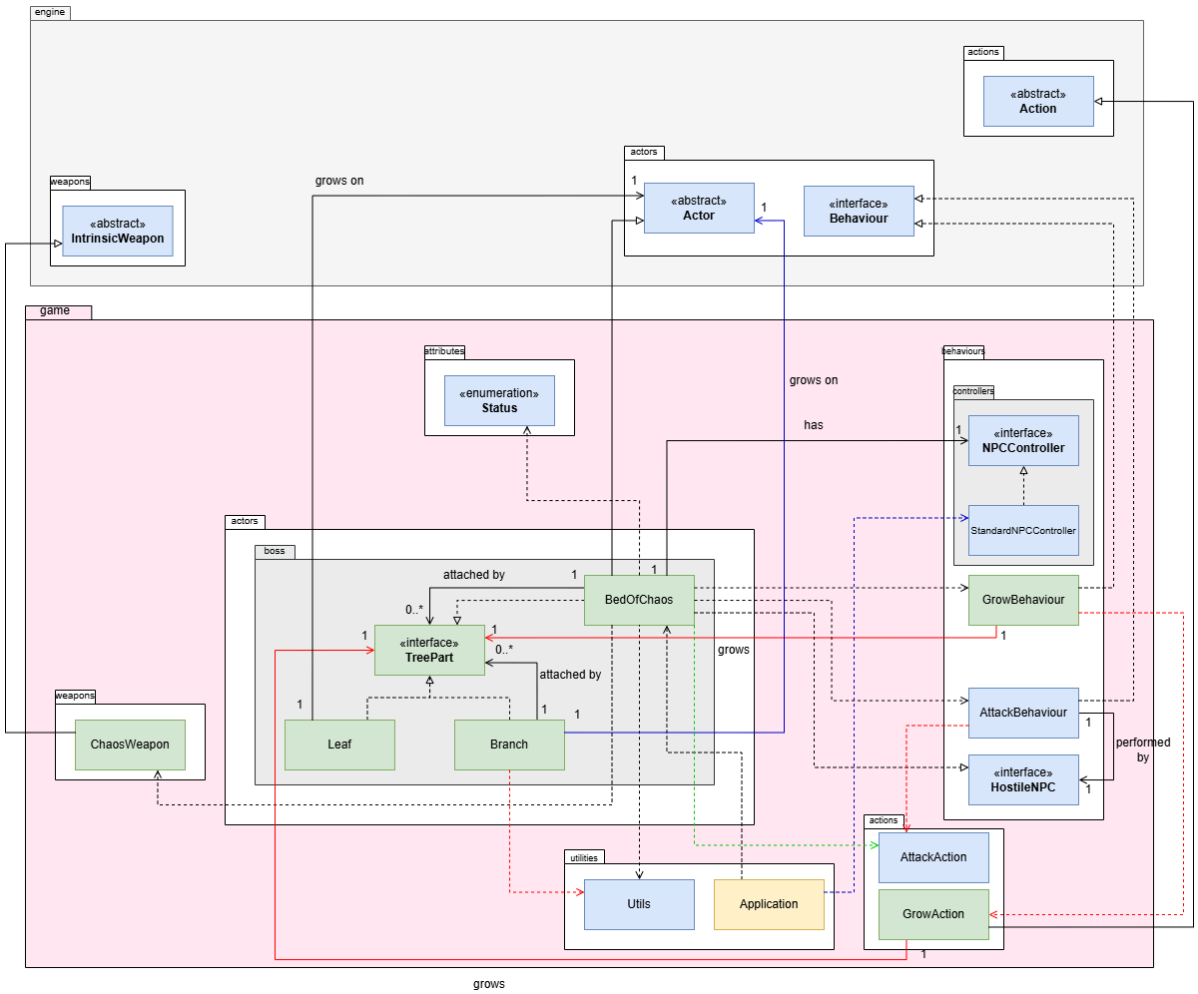
Design Documentation

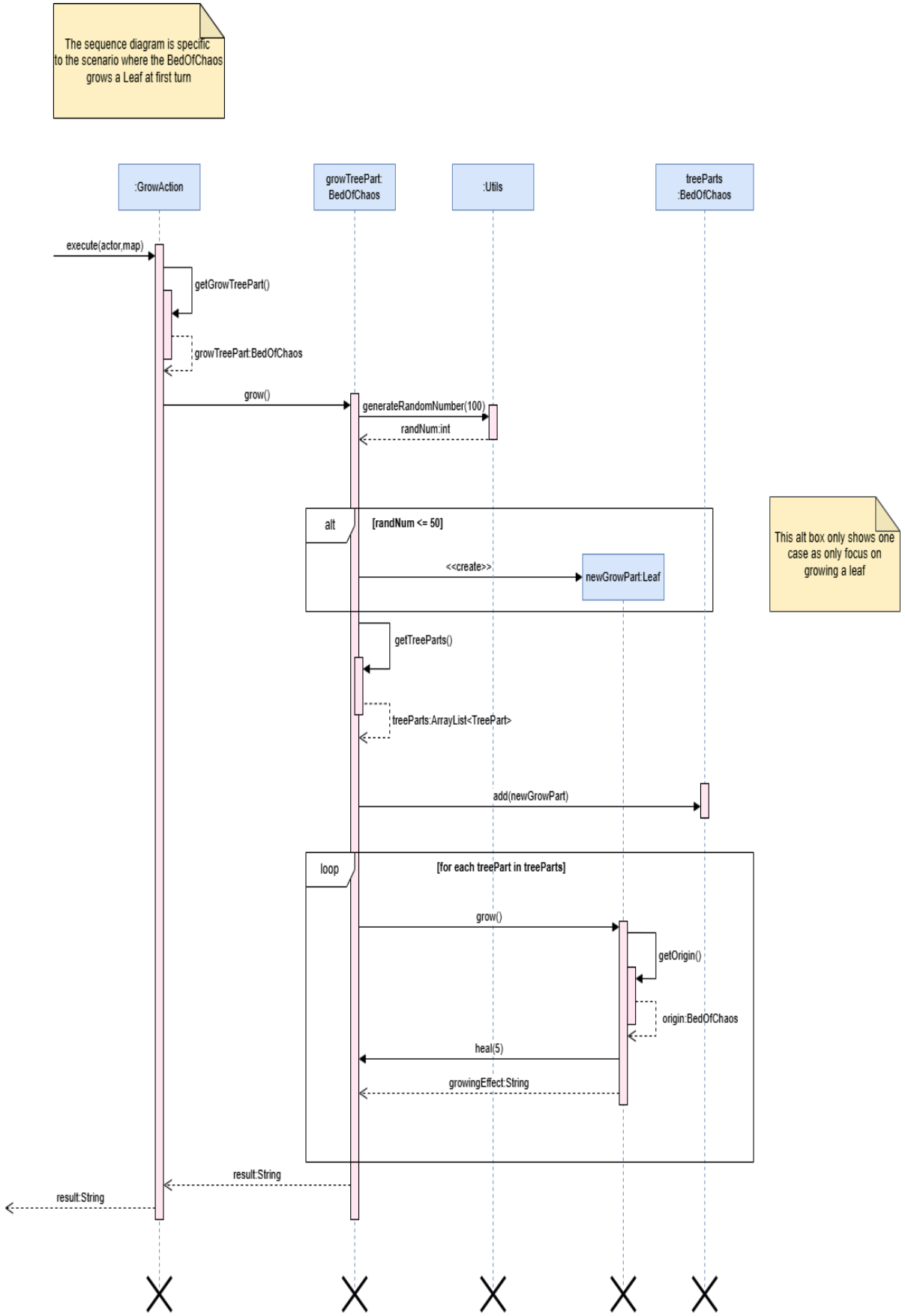
Req 2 - Bed of Chaos

UML diagram:

Colour	Indicator
Yellow	Modified class
Blue	Existing class
Green	New class



Sequence diagram:



Design Rationale:

A. Representing tree parts

Design 1: Create an abstract class TreePart, and concrete classes Leaf and Branch extends it.

Pros	Cons
Defining shared attributes, methods and methods implementation logic of tree parts, ensuring consistency and minimizing repeated code	Tighter coupling for inheritance relationship when tree parts have minimum shared method implementation logic
	Tree parts cannot extend multiple abstract classes (i.e BedOfChaos may behave as tree part, but it is also an actor)

Design 2: Create a TreePart interface, concrete classes Leaf, Branch and BedOfChaos implements it, with methods `grow()`, `getDamage()`

Pros	Cons
Loose coupling	This interface may need modification to add methods if BedOfChaos has more behaviours and the tree parts need to have specific effects for the behaviours introduced in future.
Define all methods the tree parts should implement, ensuring consistency	
Allowing all tree parts be used interchangeably	

B. BedOfChaos has accumulated attack power

TreePart has `getDamage()` to return the damage brought by the treePart.
Create a ChaosWeapon by extending IntrinsicWeapon abstract class, with:

Design 1: Constant damage point 25, override the `attack()`, type casting the attacker to TreePart to call the `getDamage()` which collects the accumulated attack points from tree parts attached to BedOfChaos.

Pros	Cons
Simple to implement	Type casting may still violate the Liskov Substitution Principle because assuming the actor type parameter surely implements

	the TreePart interface
	Minimum of code reuse, because the similar implementation logic of `attack()` is rewrote in this class with minor differences to accommodate the accumulated attack point.

Design 2: Constant damage point 25 and a new attribute DamageMultiplierProvider interface with a method `getAdditionalDamage()`, BedOfChaos implements this interface and implements this method by returning the `getDamge()`. Override `attack()` in ChaosWeapon to attack target with base damage point and additional attack points of `getAdditionalDamage()`

Pros	Cons
No type casting	May introduce overengineering
Introduce scalability, actors who have accumulated attack power intrinsic weapon can implement the interface in the future	Minimum code reuse, because the similar implementation logic of `attack()` is rewrote in this class with minor differences to accommodate the accumulated attack point.
Loose coupling	

Design 3: A constructor with int type damage. Override the `getIntrinsicWeapon()` in BedOfChaos class to always return a new ChaosWeapon instance with new damage from `getDamage()`

Pros	Cons
Code reuse, ChaosWeapon remain to use `attack()` implementation logic defined in IntrinsicWeapon, minimize repeated code.	Need to create new ChaosWeapon instance every time performing attack behaviour

C. BedOfChaos and its tree parts can grow

TreePart has `grow()`, tree parts implement it with own growing logic,
BedOfChaos and Branch have ArrayList to store tree parts attached to them, they grow a new leaf/branch and call `grow()` of their attached tree parts each time BedOfChaos grows.
Create a GrowBehaviour to manage the growing behaviour of actors, with

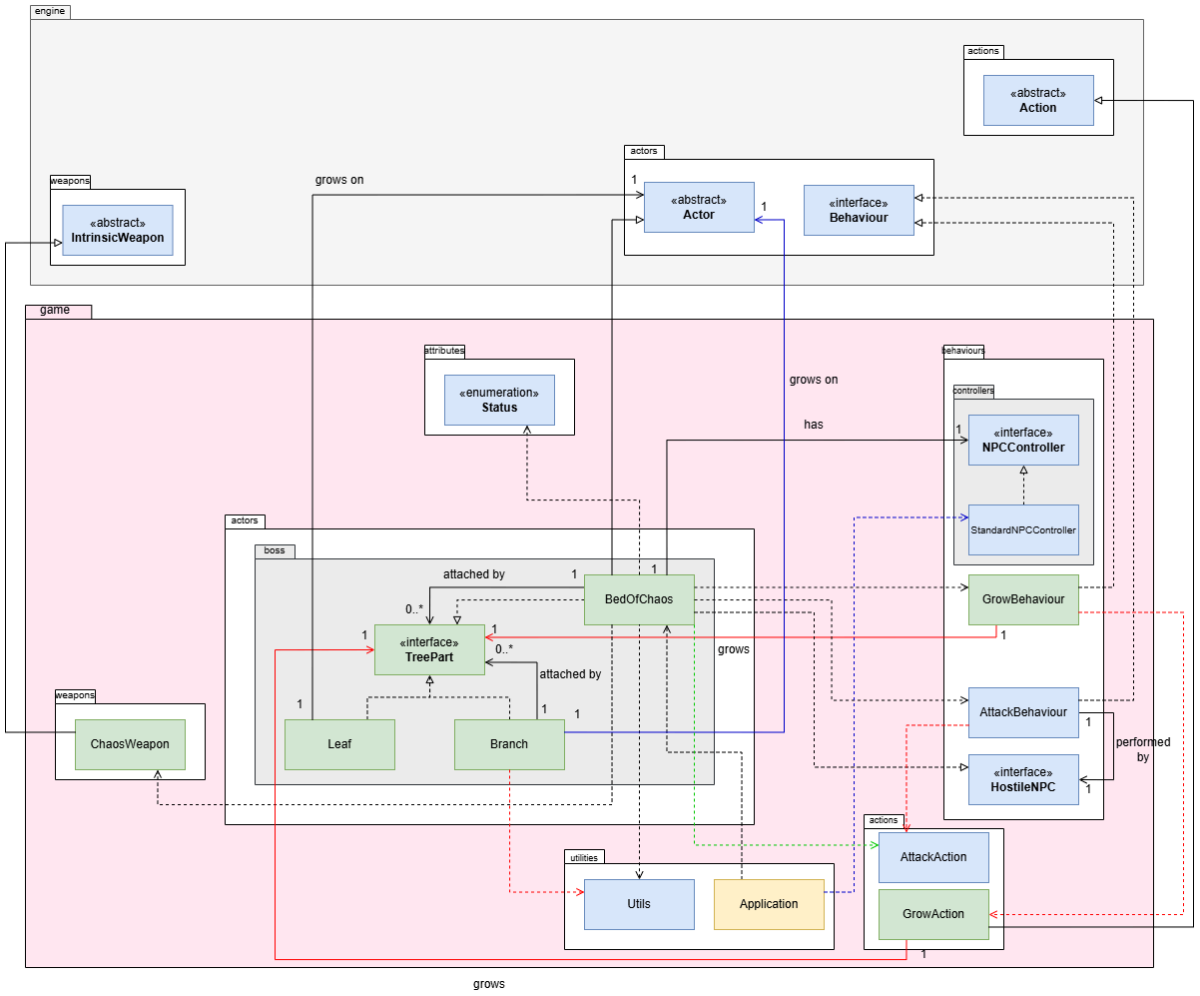
Design 1: Type casting actor parameter to TreePart in `getAction()`, calling the `grow()` and return DoNothingAction.

Pros	Cons

Simple to implement	Type casting may violate the Liskov Substitution Principle because assuming the actor type parameter surely implements the TreePart interface
	GrowBehaviour may take additional responsibility to handle growing action, instead of only handling the behaviour of returning an action for growing

Design 2: An attribute TreePart type representing the growing entity. Create a GrowAction to handle growing action. `getAction()` returns the GrowAction.

Pros	Cons
Segregating additional responsibility to GrowAction, so existing class no take additional responsibility	May introduce overengineering



The diagram above shows the final design of requirement 2, which utilizes Design 2 (part A, C) and Design 3 (part B).

The new GrowAction extends the Action abstract class, BedOfChaos extends the Actor abstract class and the ChaosWeapon extends the IntrinsicWeapon abstract class. These inheritance introduce code reuse to reduce repeated code (DRY), because these concrete classes inherit their methods and attributes from the abstract classes given in the engine.

Moreover, since each tree part has their own base damage point, instead of putting them as a magic number in the `getDamage()`, a constant static attribute is created to minimize connascence of meaning.

GrowBehaviour is created to segregate the responsibility of handling the logic of processing and returning the suitable action for an actor to grow. GrowAction is created to segregate the responsibility of handling the growing action logic. This ensures that the existing classes no need to take additional responsibility (Single Responsibility Principle).

Besides, a TreePart interface is created to define the methods that should be implemented by specific tree parts with loose coupling. This interface is implemented by concrete classes, Leaf, Branch and BedOfChaos. Thus, other classes which require to use tree parts, only need to depend on TreePart interface, a high-level class, they can use any classes that implement the TreePart interface without knowing the specific type (Dependency Inversion Principle). All classes that implement the TreePart interface can then be used interchangeably without impacting the usage in classes that use them (i.e. BedOfChaos, GrowAction), thus adhering to the Liskov Substitution Principle. When more tree parts are introduced in future, they can just implement the TreePart interface, thus no modification is needed in the existing classes that use these tree parts (Open-Closed Principle). Although this design may have a limitation, which is that the modification is needed in TreePart to add more methods if BedOfChaos has more behaviours, the tree parts attached to it may need to have specific effects for these behaviours as well. However, the current design is to avoid unnecessary complexity, the current design supports a extensibility from the perspective of allowing the BedOfChaos to grow more different tree parts, which they can provide their own growing and attack contribution by simply implementing the TreePart interface. If from a perspective of BedOfChaos has more behaviour in future, some refactoring can still be made, and since interface is used to define the contract set of methods, all classes implement it need to implement the new methods as well for consistency, thus this ensures proper maintainability, because can properly avoid the missing implementation logic of certain tree parts for new behaviour, because TreePart proper define all methods required.